

Programmazione 2

Appunti del Corso

2026

1 Introduzione: Dal "Small" al "Large"

La prima lezione segna il passaggio fondamentale tra due approcci alla programmazione:

- **Programming in the Small:** Focus su piccole procedure e familiarizzazione con il codice (Programmazione 2) [1].
- **Programming in the Large:** Focus sulla costruzione di sistemi complessi che richiedono:
 - **Divide et impera:** Suddivisione del lavoro tra più persone o gruppi [1].
 - **Manutenibilità:** Capacità di modificare il codice nel tempo senza romperlo [1].
 - **Robustezza:** Resistenza agli errori e ai casi limite [1].

Il focus principale del corso sarà il supporto fornito dai linguaggi ad oggetti (**Object-Oriented**) per gestire la programmazione "in the large" [1].

2 La Notional Machine

Un concetto pedagogico fondamentale è la **Notional Machine**. Essa è l'astrazione mentale che il programmatore crea per comprendere la **semantica** di un linguaggio, ovvero come il runtime gestisce il codice [2].

3 Gestione della Memoria e Variabili

3.1 Visibilità e Shadowing

Le variabili possono avere diversi livelli di visibilità [2, 3]:

- **Globali:** Dichiarate fuori dalle funzioni, accessibili ovunque (es. `int vg;`) [3].
- **Locali:** Visibili solo all'interno della funzione o del blocco in cui sono dichiarate [2, 4].
- **Shadowing:** Si verifica quando una variabile locale ha lo stesso nome di una globale, rendendo quest'ultima non raggiungibile in quel contesto [4].

3.2 Lo Stack e i Frame

L'allocazione della memoria per le funzioni avviene tramite gli **allocation records** (o stack frames) [5, 6]:

- Ogni chiamata a funzione crea un frame nello stack che contiene i parametri e le variabili locali [6].
- Nelle funzioni **ricorsive** (come il calcolo del fattoriale), ogni chiamata genera un nuovo frame, permettendo alla funzione di mantenere stati diversi per ogni livello di ricorsione [3, 5].

3.3 Puntatori e Memoria Dinamica

I puntatori permettono di manipolare direttamente gli indirizzi di memoria [7]:

- **Passaggio per puntatore:** Consente a una funzione di modificare il valore di una variabile definita in un altro frame (es. `incrementa_ptr(&x)`) perché la variabile originale è ancora "viva" sullo stack [5, 6].
- **Allocazione Statica:** Gestita automaticamente dal compilatore (es. `int x;`) [5].
- **Allocazione Dinamica:** Gestita manualmente tramite **new** (per creare spazio nell'**heap**) e **delete** (per liberarlo) [7].

4 Esempio di Codice: Puntatori e Memoria

```
void test_mem_dyn() {  
    int* px;  
    px = new int;           // Allocazione dinamica  
    *px = 3;                // De-referenziazione  
    delete(px);            // De-allocazione obbligatoria  
}
```

Listing 1: Esempio di allocazione dinamica dalla lezione

5 Lezione 2: Introduzione al Paradigma Object-Oriented

Il passaggio al **Programming in the Large** richiede un approccio modulare per gestire l'evoluzione del codice e il lavoro di gruppo . Il paradigma ad oggetti (OO) fornisce gli strumenti necessari per massimizzare il **riuso del codice** e minimizzare gli errori .

5.1 Concetti Fondanti dell'OO

Il paradigma si basa su cinque pilastri principali :

- **Oggetti:** Entità dotate di **stato** (dati), **comportamento** (metodi), un identificativo univoco e una locazione in memoria .
- **Incapsulamento:** Protezione dello stato interno tramite interfacce e *information hiding* .
- **Classificazione:** Organizzazione degli oggetti in classi tramite ereditarietà .
- **Specializzazione:** Capacità di estendere classi esistenti aggiungendo nuovi comportamenti o stati .
- **Polimorfismo:** Capacità di un oggetto di essere utilizzato in contesti di tipo differente .

5.2 Terminologia Tecnica

- **Classe:** Uno "stampino" (blueprint) che definisce la forma dello stato (*memory footprint*) e il comportamento comune a tutte le sue istanze .
- **Oggetto:** Un'istanza specifica di una classe con il proprio stato .
- **Campo (Variabile d'istanza):** Una componente dello stato di un oggetto .
- **Metodo:** Comportamento invocabile su un oggetto .
- **Firma (Signature):** Definizione dei tipi di input e output di un metodo .

5.3 Il linguaggio Java

Java è scelto per la sua natura **fortemente tipata** e modulare . In Java, tutto risiede all'interno di una classe; non esistono metodi "sciolti" .

- **Metodo main:** Punto di ingresso del programma con firma `public static void main(String[] args)` .
- **Tipi:** Distinguiamo tra **Tipi Primitivi** (`int`, `boolean`, `char`) e tipi definiti dall'utente (Classi) per modellare domini specifici come Minecraft .
- **Enums:** Tipi speciali per definire set fissi di costanti (es. `Direction.NORTH`, `ToolTier`) .

6 Lezione 3: Organizzazione, Visibilità e Stato Statico

La terza lezione approfondisce come strutturare grandi progetti e come gestire l'accesso ai dati.

6.1 Package e Struttura

I **Package** sono strumenti semantici utilizzati per fare ordine in progetti complessi, raggruppando classi correlate (es. `lecture03.ackages.blocks`) . Il nome completo di una classe include il percorso del suo package .

6.2 Modificatori di Accesso (Incapsulamento)

L'incapsulamento viene implementato tramite parole chiave che regolano la visibilità :

- **public**: La classe o il membro è visibile a tutto il progetto .
- **private**: Il membro è visibile solo all'interno della propria classe (essenziale per proteggere l'integrità dei dati, come la vita di un giocatore) .
- **Default (C++)**: A differenza di Java, in C++ i campi sono privati di default .

6.3 Membri Statici vs Istanza

- **Campi d'istanza**: Ogni oggetto ha la sua copia (es. ogni TNT ha il suo `isIgnited`; innescarne una non innesca le altre) .
- **Campi statici**: Appartengono alla classe e sono condivisi da tutte le istanze. Vengono spesso usati per costanti globali del gioco (`GameConstants`) .

6.4 Modificatore Final e Singleton

- **final**: Una variabile dichiarata `final` non può essere modificata dopo l'inizializzazione (es. il nome di un giocatore immutabile) .
- **Singleton Pattern**: Un design pattern utilizzato quando deve esistere **una sola istanza** di una classe in tutto il sistema (es. `DragonEgg` in Minecraft) .

7 Lezione 4: Incapsulamento, Invarianti e Gestione Memoria

La quarta lezione approfondisce come proteggere l'integrità dei dati e come il linguaggio gestisce gli oggetti in memoria.

7.1 Incapsulamento e Invarianti

L'obiettivo dell'incapsulamento non è solo nascondere i dati, ma garantire il rispetto delle **invarianti** .

- **Invariante:** Una condizione che deve essere sempre vera per un oggetto durante tutto il suo ciclo di vita (es. la vita di un **Player** non può essere negativa o superare il massimo).
- **Protezione:** Rendendo i campi **private**, obblighiamo l'uso di metodi (getter/setter) che validano i dati prima di modificare lo stato, impedendo stati inconsistenti .

7.2 Passaggio dei Valori

Un concetto spesso frainteso in Java è come vengono passati i dati alle funzioni :

- **Tipi Primitivi:** Viene passata una copia del valore. La modifica del parametro non influisce sulla variabile originale.
- **Oggetti:** Viene passata una copia del **riferimento** (indirizzo di memoria). Questo significa che se modifichiamo lo stato dell'oggetto all'interno di un metodo, la modifica è visibile ovunque perché l'oggetto è lo stesso, ma non possiamo far puntare il riferimento originale a un nuovo oggetto.

7.3 Layout della Memoria

La lezione introduce la distinzione tra come le informazioni sono organizzate:

- **Class Memory Layout:** Come la classe definisce la struttura dei dati.
- **Object Memory Layout:** Come un'istanza specifica occupa effettivamente spazio nell'Heap .

8 Lezione 5: Ereditarietà e Gerarchie di Tipi

La quinta lezione introduce l'ereditarietà come strumento per la modularizzazione e il riuso del codice .

8.1 Ereditarietà (Extends)

L'ereditarietà permette di definire una relazione di tipo "*is-a*" (un oggetto B è un tipo di oggetto A) .

- **Superclasse (Entity):** Contiene i campi e i metodi comuni a tutte le entità (es. posizione, salute) .

- **Sottoclassi (Zombie, Creeper):** Estendono la superclasse (`extends Entity`) ereditandone le caratteristiche e aggiungendo comportamenti specifici (es. l'esplosione per il Creeper) .

8.2 Vantaggi dell'Ereditarietà

- **Generalizzazione:** Fattorizzare il codice comune verso l'alto nella gerarchia per evitare duplicazioni .
- **Specializzazione:** Permettere a tipi diversi di rispondere in modo specifico a determinati stimoli pur essendo trattati come tipi generici .

8.3 Il Problema del Diamante (Diamond Problem)

Il **Diamond Problem** è un'ambiguità che si verifica nell'ereditarietà multipla (quando una classe eredita da due classi che hanno un antenato comune)

- **Soluzione Java:** Per evitare conflitti e complessità nel runtime, Java **non permette l'ereditarietà multipla tra classi** (una classe può estendere una sola superclasse) . Vedremo più avanti come le *interfacce* risolvono parte di questa necessità in modo sicuro.

9 Lezione 6: Classi Astratte, Overloading e Overriding

La sesta lezione approfondisce come raffinare le gerarchie di classi per gestire comportamenti specifici e modelli più complessi.

9.1 Classi Astratte

Le classi **astratte** (definite con la keyword `abstract`) fungono da modelli parziali per altre classi.

- **Non istanziabili:** Non è possibile creare un oggetto direttamente da una classe astratta (es. non si può creare un "Blocco" generico, ma solo un tipo specifico come `Dirt`).
- **Scopo:** Definire uno stato e un comportamento comune a tutte le sottoclassi, forzando al contempo una struttura coerente (es. le classi `Block` ed `Entity`).

- **Metodi Astratti:** Possono dichiarare firme di metodi senza implementazione, obbligando le sottoclassi a fornirne una specifica.

9.2 Overloading (Sovraccarico)

L'overloading si verifica quando all'interno della stessa classe (o in una gerarchia) esistono più metodi con lo stesso nome ma firme differenti.

- **Firma:** Cambia il numero, il tipo o l'ordine dei parametri.
- **Esempio:** Una classe `Pick` (piccone) potrebbe avere diversi metodi per scavare tipi di oggetti diversi, come `Chicken`, `Dirt` o `Bedrock`.

9.3 Overriding (Sovrascrittura)

L'overriding avviene quando una sottoclasse fornisce una nuova implementazione di un metodo già definito nella sua superclasse.

- **Scopo:** Permettere a un oggetto di rispondere in modo specifico a un'invocazione di metodo (es. il comportamento di `Andesite`, `Bedrock` o `GoldBlock` quando vengono interagiti).
- **Annotazione `@Override`:** In Java, viene usata per indicare esplicitamente che si sta sovrascrivendo un metodo della superclasse, aiutando il compilatore a rilevare errori di battitura nelle firme.

9.4 Strutture Dati Interne: Enums

Le classi possono contenere tipi enumerati (**Enums**) per definire set di costanti strettamente legate alla classe stessa (es. `Block.Material` che include `DIRT`, `BEDROCK`, ecc.).